Autonomous Data Intelligence Agent

Technical Design Document | v1.0 | April 2025

For internal developer use

1 Abstract

ADIA is a tool-augmented LLM agent that lets users query a relational database, get analytical insights, trigger automated actions, and receive chart-based explanations — all via natural language. It combines an LLM planning layer with deterministic backend tools so responses are grounded in real data, not model memory.

2 Introduction

Problem. Traditional analytics requires manual SQL, static dashboards, and human interpretation. Non-technical users cannot self-serve; analysts become bottlenecks.

Solution. ADIA acts as a *Digital Employee*: it understands intent, queries the database, reasons about the results, triggers business actions, and explains findings in plain English — autonomously.

Core capabilities:

- Natural language → SQL query execution
- Insight generation (trend analysis, anomaly detection)
- Autonomous action triggering (alerts, conditional rules)
- Chart generation with plain-English explanation
- Session-based conversational memory
- Retrieval-Augmented Generation (RAG) over business documents

3 System Overview & Architecture

ADIA is a **five-layer system**. The LLM only plans and synthesises; all data access and computation is done by deterministic tools.

```
[ Client / REST API  (FastAPI)          ]
         |
[ Agent Layer  (LLM Planner)            ]
         |
[ Tool Layer                            ]
  SQL Engine | Insight | Action | Viz | RAG
         |
[ Memory / Context Layer                ]
  Session Store  |  FAISS Vector Index
         |
[ Database Layer                        ]
  SQLite (structured)  |  Document Store
```

Request lifecycle:

1. Client sends natural language query via `POST /query`.
2. API loads session history and injects DB schema into the LLM prompt.
3. LLM classifies intent and produces an ordered tool-execution plan.
4. Tools execute (SQL → Insight → Action → Viz → RAG as needed).

5. LLM synthesises all tool outputs into a final response.
6. Interaction is saved to session memory; JSON response is returned.

4 Core Components

4.1 API Layer — FastAPI

Stateless REST interface. Validates input via Pydantic, routes to the Agent Layer.

Endpoint	Purpose
`POST /query`	Submit a natural language query
`POST /action`	Define a conditional business rule
`POST /visualise`	Request a chart for a query
`GET /history`	Fetch session conversation history
`POST /rag/ingest`	Upload documents for RAG indexing

4.2 Agent Layer — LLM Orchestrator

Uses the **ReAct** (Reasoning + Acting) pattern. At each step the LLM produces a *Thought*, selects an *Action* (tool call), observes the result, and decides whether to call another tool or return a final answer.

```
def agent_loop(query, context, tools):
    prompt = build_prompt(query, context.history, tool_descriptions)
    for _ in range(MAX_STEPS):
        thought, action = llm.plan(prompt)
        if action.type == "FINISH":
            return action.answer
        observation = tools[action.name].run(action.args)
        prompt = extend_prompt(prompt, thought, action, observation)
```

4.3 SQL Execution Engine

Generates SQL from natural language, validates it (read-only), executes against SQLite, and retries up to 2 times on error by feeding the error back to the LLM.

Safety: Only `SELECT` allowed. Row cap: 10,000. Timeout: 30s.

4.4 Insight Engine

Runs multiple SQL queries and computes: period-over-period deltas, Z-score anomalies, and comparative rankings. Uses `pandas` and `scipy`. Invoked when intent is *analytical* rather than just *retrieval*.

4.5 Action Engine

Evaluates user-defined rules: fetch metric → compare to threshold → execute action. Conditions are evaluated by Python, never the LLM.

```
# Example rule
{ "metric": "monthly_revenue", "operator": "<",
  "threshold": 50000, "action": "alert" }
```

4.6 Visualisation Engine

Picks an appropriate chart type (line, bar, pie, scatter), renders it with **Matplotlib**, and returns the image path alongside a 2–3 sentence LLM-generated explanation.

4.7 RAG Retriever

Documents are chunked → embedded (sentence-transformers) → stored in FAISS. On query, the top-5 nearest chunks are retrieved and injected into the LLM prompt as supporting context.

4.8 Memory / Context Layer

In-memory session dictionary keyed by session UUID. Stores the last $N = 10$ turns. Older turns are compressed into a summary to stay within the LLM token budget.

5 Agent Workflow

1. **Intent classification** — LLM labels the query as one of: `DATA_RETRIEVAL`, `INSIGHT_ANALYSIS`, `ACTION_TRIGGER`, `VISUALISATION`, `RAG_QUERY`.
2. **Plan generation** — LLM outputs an ordered list of tool calls.
3. **Tool execution loop** — Tools run sequentially; each output is appended to the context before the next step.
4. **Response synthesis** — LLM combines all observations into a plain-English answer with numbers, trends, and action outcomes.
5. **Persist** — Query + response saved to session store.

6 Example Use Case

Query: *"Why did revenue drop last month?"*

Step 1 — SQL Engine runs two queries:

```sql
-- Current month
SELECT SUM(revenue) FROM sales WHERE sale_date BETWEEN '2025-03-01' AND '
    2025-03-31';
-- Result: 128400

-- Previous month
SELECT SUM(revenue) FROM sales WHERE sale_date BETWEEN '2025-02-01' AND '
    2025-02-28';
-- Result: 136900
```

Step 2 — Insight Engine computes: overall drop = \$8,500 (−6.2%). Breakdown by category reveals Electronics fell 24.2% — primary driver.

Step 3 — RAG Retriever finds a passage in the Q1 Board Report citing supply chain delays and a new competitor launch in Electronics.

Final response:

```
Revenue fell 6.2% in March vs February (-$8,500).
Primary cause: Electronics category dropped 24.2% (-$13,100).
Factors (from Board Report): supply chain delays on X12 Pro
model + competitor launch at 15% lower price on March 5th.
Recommendation: targeted Q2 promotional campaign for Electronics.
```

7 Technology Stack

Component	Technology	Why
API framework	FastAPI	Async, auto-docs, Pydantic validation
LLM layer	LangChain	Model-agnostic, tool-use abstractions
Database	SQLite	Zero-config, portable, SQL-92
Vector store	FAISS	Fast nearest-neighbour search
Embeddings	sentence-transformers	Local, no API cost
Data analysis	pandas + scipy	DataFrames, stats, anomaly detection
Visualisation	Matplotlib	Programmatic, full layout control
Validation	Pydantic v2	Schema enforcement at API boundary
Language	Python 3.11+	Ecosystem, async support

8 Design Principles

- **LLM plans; tools compute.** All arithmetic, SQL execution, and condition checks are done by deterministic code — never by the LLM. This eliminates numerical hallucination.
- **Read-only safety.** The SQL Engine rejects any non-`SELECT` statement before execution.
- **Modularity.** Each tool exposes one `run()` method and has no knowledge of other tools — independently testable and swappable.
- **Graceful degradation.** On tool failure the agent returns a structured error rather than fabricating an answer.
- **Stateless API.** The API layer holds no state; session state lives in the Memory Layer, enabling horizontal scaling behind a load balancer.

9 Limitations

- **LLM SQL errors:** The model may generate valid SQL with wrong semantics (bad join key, wrong date range). Mitigated by schema injection and error-retry loops.
- **SQLite concurrency:** Write-locking limits multi-user throughput. Migrate to PostgreSQL for production.
- **In-memory session store:** Sessions lost on restart. Replace with Redis for persistence.
- **FAISS flat index:** $O(n)$ search degrades on large corpora; switch to IVF/HNSW for >100K chunks.
- **Action Engine v1.0:** Supports log and in-memory alert only; email/Slack planned for v1.1.

10 Future Enhancements

- **v1.1:** React dashboard UI; email/Slack/webhook alerts; scheduled rule evaluation.
- **v1.2:** Kafka/Kinesis streaming data ingestion for real-time metrics.
- **v2.0:** Multi-agent architecture (coordinator + specialist sub-agents); forecasting engine (Prophet/ARIMA); multi-database federation via DuckDB.

11 Conclusion

ADIA transforms a passive relational database into an intelligent, conversational analytics system. The separation of LLM reasoning from deterministic tool execution is its key engineering decision: it makes the system reliable enough to trust with business-critical queries while remaining flexible enough to handle open-ended natural language. The modular architecture means any layer — the LLM provider, the database, the vector store — can be swapped without rebuilding the system.

ADIA TDD v1.0 | System Architecture Team | April 2025